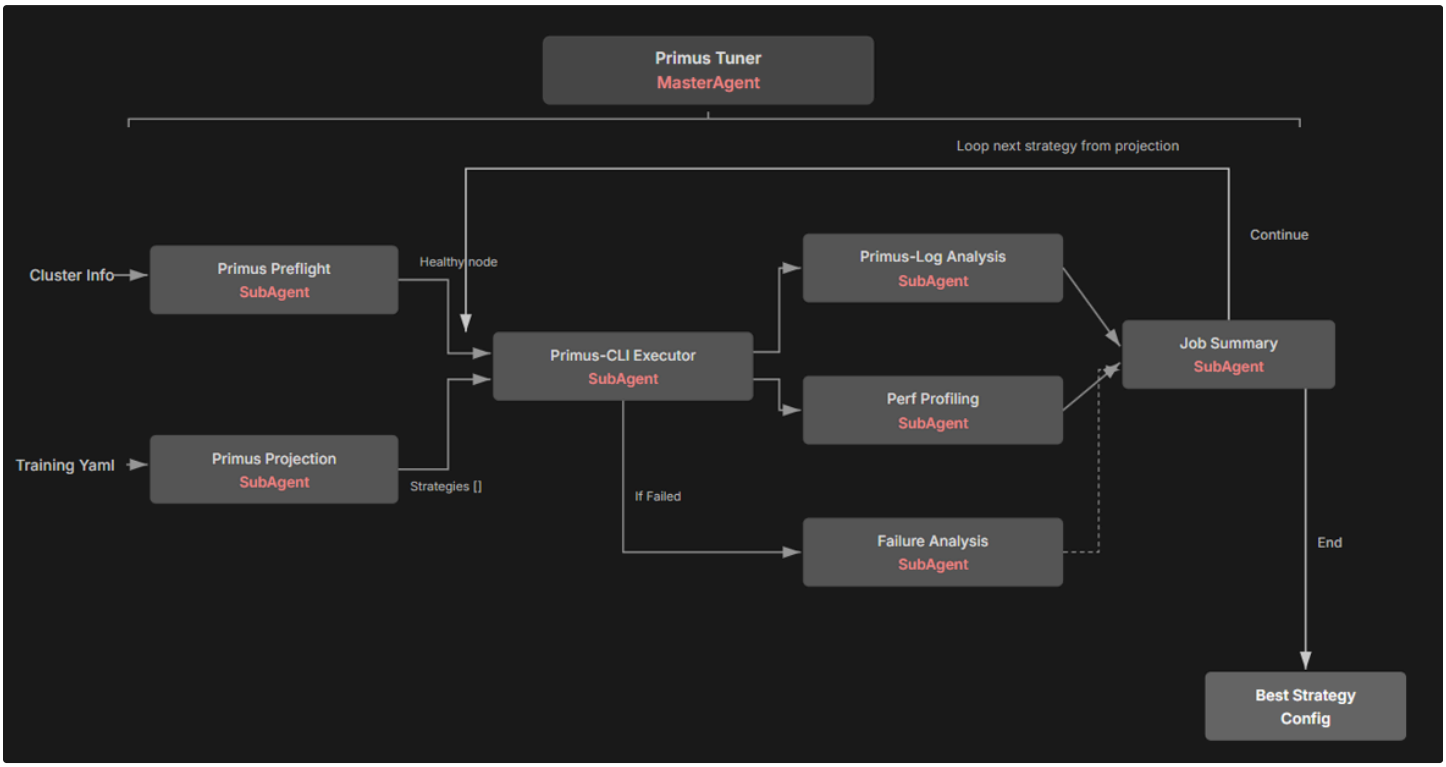
Primus-Tuner Agent

master-agent-... ph.html
15 Apr 2026, 10:54 PM



Primus Model Tuning Master Agent — Complete Design

Based on deep analysis of the Primus project architecture (supporting Megatron-LM / TorchTitan / JAX MaxText, SLURM/Container/Direct deployment modes, YAML configuration system), I've designed a **6-phase, 14 Sub-Agent** orchestration system.

Overall Architecture: Master Agent (Tuning Orchestrator)

```
User Goal Input (model / hardware / performance target)
          │
      ┌───▼────┐
      │ MASTER │   State Machine · Event-Driven · Retry-Aware
      │ AGENT  │   Receive goal → Orchestrate Sub-Agent chain → Closed-loop iteration
      └────────┘
          │
═══════════════════════════════════════════════════════
```

Phase 1: Pre-flight Environment Readiness `[Parallel]`

#	Sub-Agent	Description	Input / Output	Status

1	**Cluster Health Check**	SSH connectivity, node state (sinfo/scontrol), Docker env, workspace mount validation	IN: SLURM partition, node list → OUT: healthy node list + anomaly report	**Existing SKILL**
2	**Network Topology Analysis**	RDMA link status, NIC QoS/DCQCN config, GID table validation, inter-node bandwidth (ib_write_bw)	IN: candidate nodes → OUT: topology graph + bandwidth matrix + flagged nodes	Partially exists
3	**Hardware Compatibility Check**	ROCm version, GPU model (MI300X/MI355X), driver version, Docker image matching	IN: target Docker image, node list → OUT: compatibility report + recommended image	**New**

> All 3 agents run **fully in parallel**. Combined output: healthy node list + network topology + hardware profile.

Phase 2: Configuration Analysis & Optimization `[Sequential + Parallel]`

#	Sub-Agent	Description	Input / Output	Status
4	**Memory Analysis**	Estimate VRAM usage under various parallelism strategies based on model params, seq_len, batch size (parameters / gradients / optimizer states / activations / KV-cache)	IN: model YAML + training config + GPU VRAM capacity → OUT: memory breakdown report	**New · Critical**
5	**Distributed Strategy Tuning**	TP/PP/DP/EP/VPP/CP combination recommendation, Zero-Bubble Pipeline,	IN: memory analysis + hardware topology + perf target → OUT: optimal parallelism	**New · Critical**

		FSDP strategy (depends on #4)	config + estimated MFU	
6	**Mixed Precision Strategy**	FP8/BF16/FP16 selection, loss scaling config, precision-performance tradeoff analysis	IN: model type + GPU model + precision requirements → OUT: precision config + expected speedup	**New**
7	**Hyperparameter Recommendation**	Learning rate, batch size, warmup, weight decay recommendations based on model scale	IN: model scale + data scale + training token target → OUT: hyperparameter config + Scaling Law reference	**New**
8	**Data Pipeline Check**	Dataset path validation, tokenizer config, dataloader performance estimation	IN: data paths + tokenizer config from training YAML → OUT: data readiness status + estimated IO bottleneck	**New**

> Execution order: #4 runs first (memory analysis) → #5 depends on #4 output → #6 / #7 / #8 run in parallel.

Decision Gate: Does the config pass memory safety check?

```
┌──────────────────────────────────────┐
│  Estimated VRAM < GPU capacity × 0.9 ? │
└──────────────┬──────────────────────┘
    YES │              │ NO
        ▼              ▼
Continue to        Fall back to Phase 2
Phase 3            (increase TP/PP, enable
                    activation checkpointing,
                    reduce batch size)
```

Phase 3: Training Execution `[Sequential]`

#	Sub-Agent	Description	Input / Output	Status
9	**Primus-CLI Executor**	Invoke `primus-cli` to launch training (direct/container/slur	IN: final YAML config + execution mode + node list → OUT: Job	Partially exists

		m mode), manage lifecycle, dry-run validation	ID + log path + real-time status	
10	**Node Performance Validation**	Run short smoke test on target nodes (4-node groups, detect iteration 3 completion), verify actual usability	IN: candidate nodes + test config → OUT: pass/fail node list + baseline performance data	**Existing SKILL**

> #10 serves as pre-check before full training; #9 is the core execution entry point.

Phase 4: Training Monitoring `[Continuous]`

#	Sub-Agent	Description	Input / Output	Status
11	**Log Real-time Monitor**	Parse training log stream, track loss/LR/throughput curves, detect loss spikes / NaN / gradient explosion / hang	IN: training log path + monitoring thresholds → OUT: real-time dashboard + anomaly alerts	**New · Critical**
12	**Performance Profiling**	GPU utilization, MFU, communication time ratio analysis, identify compute/communication/IO bottlenecks	IN: training process + profiling config → OUT: bottleneck report (compute/comm/IO) + optimization suggestions	**New**

> #11 runs continuously; #12 is triggered on-demand (e.g., when throughput falls below expectations).

Decision Gate: Did training complete successfully?

```
          ┌──────────────────────────────┐
          │  Training completed normally?  │
          └──────────────┬───────────────┘
            Success │          │ Failure / Anomaly
                    ▼          ▼
```

```
            Output final   Trigger Phase 5
              report        diagnostics
```

Phase 5: Failure Diagnosis & Recovery `[On-demand]`

#	Sub-Agent	Description	Input / Output	Status
13	**Training Failure Root Cause Analysis**	Parse error logs, classify fault type (OOM / NCCL timeout / NaN / Hang / Hardware failure), propose fix	IN: error logs + training config + monitoring data → OUT: root cause report + fix suggestions + auto-fix actions	**New · Critical**
14	**Checkpoint Manager**	Resume-from-checkpoint config, checkpoint resharding (on parallelism strategy change), storage cleanup policy	IN: checkpoint path + old/new parallelism strategy → OUT: resume config + checkpoint status	**New**

> After failure analysis, the system automatically determines the recovery path:
>
> - **OOM** → Trigger #4 Memory Analysis + #5 Distributed Strategy adjustment
> - **NCCL timeout** → Trigger #2 Network Topology + #1 Node replacement
> - **NaN / gradient explosion** → Trigger #6 Mixed Precision + #7 Hyperparameter adjustment
> - **Hardware failure** → Trigger #1 Health Check + #10 Node replacement validation

---

Phase 6: Closed-loop Iterative Optimization `[Auto-loop]`

```
┌──────────────────────────────────────────────────────────┐
│  ↻ Performance below target → Return to Phase 2 for tuning   │
│                                                              │
│  Master Agent auto-decides based on Profiling + Failure RCA:  │
│                                                              │
│  • OOM           → Memory Analysis + Distributed Strategy     │
│  • Low perf      → Profiling + Mixed Precision + Comm optim   │
│  • Node failure   → Health Check + Node replacement           │
│  • Convergence    → Hyperparameter tuning + LR schedule adj   │
│                                                              │
│  Exit condition: MFU ≥ target && stable for N iterations      │
└──────────────────────────────────────────────────────────┘
```

Sub-Agent Dependency Graph

```
Phase 1 (Parallel)       Phase 2                  Phase 3       Phase 4       Phase 5
┌──────────────────┐
│ ① Health Check   │─┐
├──────────────────┤ │
│ ② Network Topo   │─┼──►│ ④ Memory   │──►┌──────────────┐
├──────────────────┤ │   │   Analysis │   │ ⑤ Distrib    │
│ ③ HW Compat      │─┘   └────────────┘   │   Strategy   │──►┌──────────┐   ┌──────────┐   ┌──────────┐
                                           └──────────────┘   │ ⑨ CLI    │──►│ ⑪ Log    │──►│ ⑬ Failure│
                          ┌────────────┐                      │   Exec   │   │  Monitor │   │   RCA    │
                          │ ⑥ Mixed    │──────────┐           ├──────────┤   ├──────────┤   ├──────────┤
                          │   Precision│          │           │ ⑩ Node   │   │ ⑫ Perf   │   │ ⑭ Ckpt   │
                          ├────────────┤          │           │   Valid  │   │  Profile │   │  Manager │
                          │ ⑦ Hyper-   │──────────┼──► [Generate Final YAML]  └──────────┘   └──────────┘
                          │   params   │          │           └──────────┘
                          ├────────────┤          │
                          │ ⑧ Data     │──────────┘
                          │   Pipeline │
                          └────────────┘

                                      ◇ Memory Safety Gate
                                   YES│        │NO → Fallback to Phase 2
```

Mapping to Existing Primus Components

Sub-Agent	Corresponding Primus Component
Cluster Health Check	`slurm-idle-node-check` SKILL
Node Performance Validation	`slurm-training-node-validation` SKILL
CLI Executor	`runner/primus-cli` + `primus/cli/subcommands/train.py`
Distributed Strategy	`primus/configs/models/megatron/` (TP/PP/EP params)
Mixed Precision	`examples/megatron/configs/MI355X/deepseek_v3-FP8-pretrain.yaml` etc.
Log Monitor	`primus/core/runtime/logging.py` + `primus/backends/megatron/patches/training_log/`
Checkpoint Manager	Checkpointing config in `primus/configs/modules/megatron/trainer_base.yaml`

Suggested Implementation Priority

Priority	Sub-Agents	Rationale
P0 — Core Loop	Memory Analysis → Distributed Strategy Tuning → CLI Executor → Failure RCA	Minimum viable closed-loop: analyze → execute → diagnose → retry

P1 — Monitoring	Log Real-time Monitor → Performance Profiling → Checkpoint Manager	Enables proactive issue detection and seamless recovery
P2 — Full Coverage	Hardware Compatibility → Data Pipeline → Mixed Precision → Hyperparameter Recommendation	Completes the pre-flight and config optimization phases

---

This design covers the full training lifecycle — from environment pre-flight through configuration optimization, execution, monitoring, and failure diagnosis — forming a closed-loop system. Each Sub-Agent has well-defined input/output interfaces, and the Master Agent drives the entire workflow through a state machine. Want me to start implementing any specific Sub-Agent?